

# Deploy Your Streamlit Project Using GitHub & Streamlit Cloud

- **Name of Student :** Shruti Dnyaneshwar Darwatkar
- **Domain Name:** AIML Development
- **College Name:** Zeal Polytechnic , Narhe
- **Branch Name:** AIML
- **Github url:** <https://github.com/shrutidarwatkar-sketch/Titanic-Survival-Prediction>  
**Streamlit App Link :** <https://titanic-survival-prediction-agdmsvvsizja6waehqexrz.streamlit.app/>

## Titanic Survival Prediction Using Streamlit

❖ **Backend (Google Colab):**

### Step 1

#### **Import libraries**

```
import pandas as pd
import joblib
from sklearn.model_selection import train_test_split
from sklearn.preprocessing import LabelEncoder
from sklearn.linear_model import LogisticRegression
```

### Step 2

Load Dataset

### Step 3

## Data Cleaning

- Remove missing values
- Fill missing Age
- Fill missing Embarked



## **Step 4**

### Feature Encoding

- Sex → Male/Female → Numeric
- Embarked → C/Q/S → Numeric



## **Step 5**

### Split Dataset

- Training Data
- Testing Data



## **Step 6**

### Train Model

(Logistic Regression)



## **Step 7**

### Check Accuracy



### **Step 8**

Save

model.pkl

sex\_encoder.pkl

embarked\_encoder.pkl

**Screen shots:**

```

# =====
# Titanic Survival Prediction - Model Training
# =====

# Import Libraries
import pandas as pd
import joblib

from sklearn.model_selection import train_test_split
from sklearn.preprocessing import LabelEncoder
from sklearn.linear_model import LogisticRegression
from sklearn.metrics import accuracy_score, classification_report

# -----
# Load Dataset
# -----
from google.colab import files
uploaded = files.upload()
df = pd.read_csv("Titanic-Dataset.csv")
# -----
# Select Required Columns
# -----
df = df[[
    "Pclass",
    "Sex",
    "Age",
    "SibSp",
    "Parch",
    "Fare",
    "Embarked",
    "Survived"
]]

```

```

# -----
# Handle Missing Values
# -----
df["Age"] = df["Age"].fillna(df["Age"].median())
df["Embarked"] = df["Embarked"].fillna(df["Embarked"].mode()[0])

# -----
# Encode Categorical Data
# -----
sex_encoder = LabelEncoder()
embarked_encoder = LabelEncoder()

df["Sex"] = sex_encoder.fit_transform(df["Sex"])
df["Embarked"] = embarked_encoder.fit_transform(df["Embarked"])

# -----
# Features & Target
# -----
X = df.drop("Survived", axis=1)
y = df["Survived"]

# -----
# Split Dataset
# -----
X_train, X_test, y_train, y_test = train_test_split(
    X,
    y,
    test_size=0.20,
    random_state=42
)

# -----
# Train Logistic Regression Model
# -----
model = LogisticRegression(max_iter=1000)

```

```

model.fit(X_train, y_train)

# -----
# Prediction
# -----
y_pred = model.predict(X_test)

# -----
# Accuracy
# -----
accuracy = accuracy_score(y_test, y_pred)

print("="*40)
print("Model Accuracy :", round(accuracy*100,2),"%")
print("="*40)

print(classification_report(y_test, y_pred))

# -----
# Save Model
# -----
joblib.dump(model, "model.pkl")
joblib.dump(sex_encoder, "sex_encoder.pkl")
joblib.dump(embarked_encoder, "embarked_encoder.pkl")

print("\nModel Saved Successfully")
print("Files Generated:")
print("✓ model.pkl")
print("✓ sex_encoder.pkl")
print("✓ embarked_encoder.pkl")

```

## OUTPUT :

```

... Choose files Titanic-Dataset.csv
Titanic-Dataset.csv(text/csv) - 61194 bytes, last modified: 10/07/2026 - 100% done
Saving Titanic-Dataset.csv to Titanic-Dataset (4).csv
=====
Model Accuracy : 81.01 %
=====

```

|              | precision | recall | f1-score | support |
|--------------|-----------|--------|----------|---------|
| 0            | 0.83      | 0.86   | 0.84     | 105     |
| 1            | 0.79      | 0.74   | 0.76     | 74      |
| accuracy     |           |        | 0.81     | 179     |
| macro avg    | 0.81      | 0.80   | 0.80     | 179     |
| weighted avg | 0.81      | 0.81   | 0.81     | 179     |

```

Model Saved Successfully
Files Generated:
✓ model.pkl
✓ sex_encoder.pkl
✓ embarked_encoder.pkl

```

```
from google.colab import files

files.download("model.pkl")
files.download("sex_encoder.pkl")
files.download("embarked_encoder.pkl")
```

## ❖ Frontend (VS Code):

### Step 1: Open VS Code

Open Visual Studio Code.

### Step 2: Create a Project Folder

Create a folder named:

Titanic\_Survival\_Prediction

### Step 3: Create the Project Structure

Inside the folder, create:

Titanic\_Survival\_Prediction/

|

|— app.py

|— requirements.txt

|— README.md

|

|— Models/

|— model.pkl  
|— sex\_encoder.pkl  
└— embarked\_encoder.pkl

Copy the .pkl files that you downloaded from Google Colab into the **Models** folder.

#### **Step 4: Open the Folder**

In VS Code:

- Click **File** → **Open Folder**
- Select **Titanic\_Survival\_Prediction**

#### **Step 5: Open the Terminal**

Go to:

**Terminal** → **New Terminal**

#### **Step 6: Install Required Libraries**

Run:

```
pip install -r requirements.txt
```

## Step 7: Write the Frontend Code

Open **app.py**

Paste the Streamlit code that we created earlier.

Save the file.

## Step 8: Run the Application

In the terminal, type:

```
streamlit run app.py
```

Press **Enter**.

## Step 9: Browser Opens Automatically

Streamlit will open in your browser.

If it doesn't, open the link shown in the terminal, for example:

Local URL: <http://localhost:8501>

## Screen Shots:

```
app.py > ...
1  import streamlit as st
2  import pandas as pd
3  import joblib
4
5  # -----
6  # Page Configuration
7  # -----
8
9  st.set_page_config(
10     page_title="Titanic Survival Prediction",
11     page_icon="🚢",
12     layout="wide"
13 )
14
15 # -----
16 # Load Model
17 # -----
18
19 model = joblib.load("Models/model.pkl")
20 sex_encoder = joblib.load("Models/sex_encoder.pkl")
21 embarked_encoder = joblib.load("Models/embarked_encoder.pkl")
22
```

```
app.py > ...
23 # -----
24 # Title
25 # -----
26
27 st.title("🚢 Titanic Survival Prediction")
28 st.write("Predict whether a passenger would survive the Titanic disaster.")
29
30 st.divider()
31
32 # -----
33 # Sidebar
34 # -----
35
36 st.sidebar.title("Titanic Prediction")
37
38 st.sidebar.info(
39     """
40     Enter passenger details and click
41     Predict Survival.
42     """
43 )
44
```

```
app.py > ...
45 # -----
46 # Input Form
47 # -----
48
49 st.header("Passenger Details")
50
51 col1, col2 = st.columns(2)
52
53 with col1:
54
55     pclass = st.selectbox(
56         "Passenger Class",
57         [1,2,3]
58     )
59
60     gender = st.selectbox(
61         "Gender",
62         ["male", "female"]
63     )
64
65     age = st.number_input(
66         "Age",
67         min_value=0,
68         max_value=100,
69         value=25
70     )
```

```
app.py > ...
72     sibsp = st.number_input(
73         "Siblings / Spouses",
74         min_value=0,
75         max_value=10,
76         value=0
77     )
78
79     with col2:
80
81         parch = st.number_input(
82             "Parents / Children",
83             min_value=0,
84             max_value=10,
85             value=0
86         )
87
88         fare = st.number_input(
89             "Fare",
90             min_value=0.0,
91             value=32.20
92         )
93
94         embarked = st.selectbox(
95             "Embarked",
96             ["C", "Q", "S"]
97         )
98
```

```

app.py > ...
99  # -----
100 # Encode Inputs
101 # -----
102
103 gender = sex_encoder.transform([gender])[0]
104 embarked = embarked_encoder.transform([embarked])[0]
105
106 # -----
107 # Prediction Button
108 # -----
109
110 predict = st.button("Predict Survival")
111 # -----
112 # Prediction
113 # -----
114
115 if predict:
116     # Create input dataframe
117     input_data = pd.DataFrame({
118         "Pclass": [pclass],
119         "Sex": [gender],
120         "Age": [age],
121         "SibSp": [sibsp],
122         "Parch": [parch],
123         "Fare": [fare],
124         "Embarked": [embarked]

```

```

126     })
127
128     # Make Prediction
129     prediction = model.predict(input_data)[0]
130
131     probability = model.predict_proba(input_data)[0]
132
133     st.divider()
134
135     st.header("Prediction Result")
136
137     if prediction == 1:
138         st.success("👉 Passenger is likely to SURVIVE")
139         confidence = probability[1] * 100

```

```

140     else:
141         st.error("✗ Passenger is NOT likely to SURVIVE")
142         confidence = probability[0] * 100
143
144     st.metric(
145         label="Prediction Confidence",
146         value=f"{confidence:.2f}%"
147     )
148
149     st.divider()
150
151     st.subheader("Passenger Details")
152
153     gender_text = "Male" if gender == 1 else "Female"
154
155     embarked_dict = {
156         0: "C",
157         1: "Q",
158         2: "S"
159     }
160
161     details = pd.DataFrame({
162         "Feature": [
163             "Passenger Class",
164             "Gender",
165             "Age",
166             "Siblings/Spouses",
167             "Parents/Children",
168             "Fare",
169             "Embarked"
170         ],
171         "Value": [
172             pclass,
173             gender_text,
174             age,
175             sibsp,
176             parch,
177             fare,
178             embarked_dict.get(embarked, embarked)
179         ]
180     })
181

```

```

182     st.table(details)
183
184     st.divider()
185
186     st.subheader("Prediction Probabilities")
187
188     prob_df = pd.DataFrame({
189         "Outcome": ["Not Survived", "Survived"],
190         "Probability": [
191             round(probability[0] * 100, 2),
192             round(probability[1] * 100, 2)
193         ]
194     })
195
196     st.dataframe(prob_df, use_container_width=True)
197
198     # -----
199     # Footer
200     # -----
201
202     st.divider()
203
204     st.caption("Titanic Survival Prediction using Machine Learning")
205     st.caption("Developed by Shruti Darwatkar")

```

## OUTPUT :

### 1. App.py output

```

C:\Users\tanve\Desktop\Titanic_EDA_Streamlit>python -m streamlit run app.py

You can now view your Streamlit app in your browser.

Local URL: http://localhost:8502
Network URL: http://192.168.1.11:8502

```

## 2. Streamlit app home page

Titanic Prediction

Enter passenger details and click Predict Survival.



### Titanic Survival Prediction

Predict whether a passenger would survive the Titanic disaster.

#### Passenger Details

Passenger Class

1

▼

Parents / Children

1

-

+

Gender

female

▼

Fare

32.20

-

+

Age

18

-

+

Embarked

Q

▼

Siblings / Spouses

1

-

+

Predict Survival

Predict Survival

### Prediction Result

 Passenger is likely to SURVIVE

Prediction Confidence

92.66%

#### Passenger Details

|   | Feature          | Value  |
|---|------------------|--------|
| 0 | Passenger Class  | 1      |
| 1 | Gender           | Female |
| 2 | Age              | 18     |
| 3 | Siblings/Spouses | 1      |
| 4 | Parents/Children | 1      |
| 5 | Fare             | 32.2   |
| 6 | Embarked         | Q      |

#### Prediction Probabilities

|   | Outcome      | Probability |
|---|--------------|-------------|
| 0 | Not Survived | 7.34        |
| 1 | Survived     | 92.66       |

Titanic Survival Prediction using Machine Learning

Developed by Shruti Darwatkar

## Titanic Prediction

Enter passenger details and  
click Predict Survival.



## Titanic Survival Prediction

Predict whether a passenger would survive the Titanic disaster.

## Passenger Details

|                    |                    |
|--------------------|--------------------|
| Passenger Class    | Parents / Children |
| 2                  | 1                  |
| Gender             | Fare               |
| female             | 34.20              |
| Age                | Embarked           |
| 87                 | C                  |
| Siblings / Spouses |                    |
| 1                  |                    |
| Predict Survival   |                    |

## Prediction Result

✗ Passenger is NOT likely to SURVIVE

Prediction Confidence

56.75%

## Passenger Details

|   | Feature          | Value  |
|---|------------------|--------|
| 0 | Passenger Class  | 2      |
| 1 | Gender           | Female |
| 2 | Age              | 87     |
| 3 | Siblings/Spouses | 1      |
| 4 | Parents/Children | 1      |
| 5 | Fare             | 34.2   |
| 6 | Embarked         | C      |

## Prediction Probabilities

|   | Outcome      | Probability |
|---|--------------|-------------|
| 0 | Not Survived | 56.75       |
| 1 | Survived     | 43.25       |

Titanic Survival Prediction using Machine Learning

Developed by Shruti Darwatkar

## Titanic Prediction

Enter passenger details and click Predict Survival.



## Titanic Survival Prediction

Predict whether a passenger would survive the Titanic disaster.

## Passenger Details

|                                   |                    |
|-----------------------------------|--------------------|
| Passenger Class                   | Parents / Children |
| 3                                 | 0                  |
| Gender                            | Fare               |
| male                              | 34.20              |
| Age                               | Embarked           |
| 87                                | S                  |
| Siblings / Spouses                |                    |
| 0                                 |                    |
| <button>Predict Survival</button> |                    |

## Prediction Result

✗ Passenger is NOT likely to SURVIVE

Prediction Confidence

97.88%

## Passenger Details

|   | Feature          | Value |
|---|------------------|-------|
| 0 | Passenger Class  | 3     |
| 1 | Gender           | Male  |
| 2 | Age              | 87    |
| 3 | Siblings/Spouses | 0     |
| 4 | Parents/Children | 0     |
| 5 | Fare             | 34.2  |
| 6 | Embarked         | S     |

## Prediction Probabilities

|   | Outcome      | Probability |
|---|--------------|-------------|
| 0 | Not Survived | 97.88       |
| 1 | Survived     | 2.12        |

Titanic Survival Prediction using Machine Learning

Developed by Shruti Darwatkar

### ❖ Files Description:

| File                 | Description                          |
|----------------------|--------------------------------------|
| Titanic-Dataset.csv  | Dataset used for training            |
| train_model.py       | Trains the Logistic Regression model |
| model.pkl            | Trained ML model                     |
| sex_encoder.pkl      | Encoder for the Sex column           |
| embarked_encoder.pkl | Encoder for the Embarked column      |
| app.py               | Streamlit application for prediction |
| requirements.txt     | List of required Python libraries    |
| README.md            | Project documentation                |

### Complete Workflow:

Titanic Dataset (.csv)

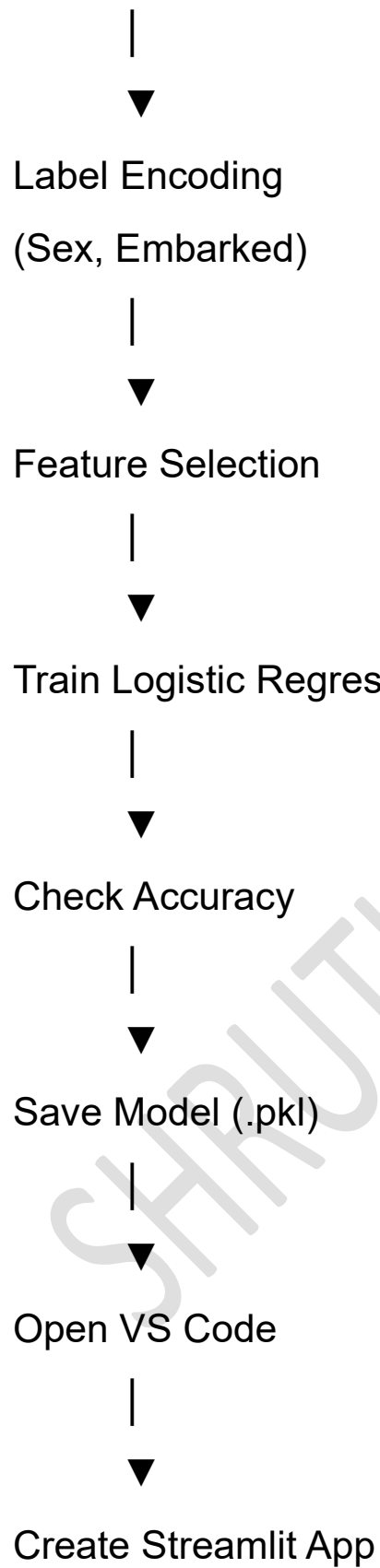


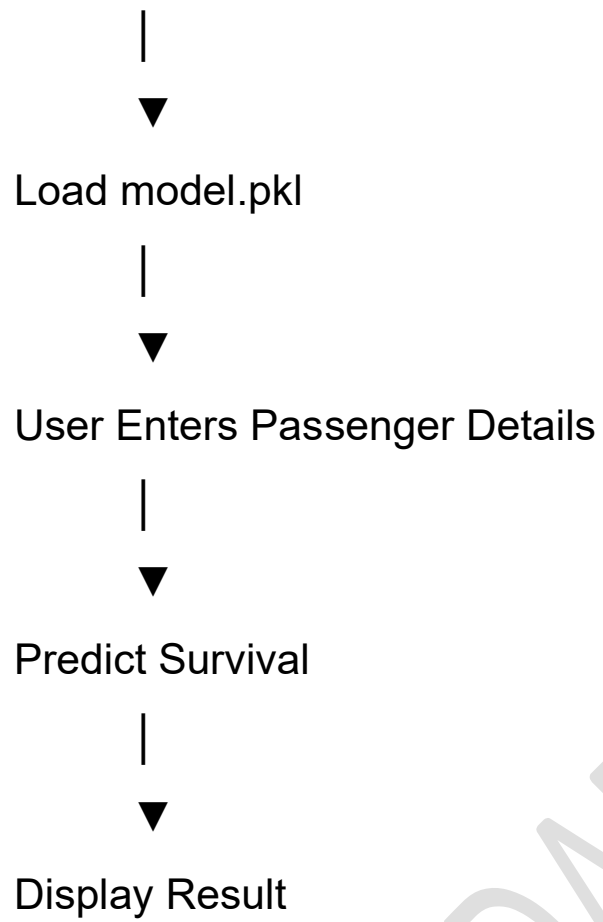
Load Dataset using Pandas



Data Cleaning

(Age, Embarked Missing Values)





## Conclusion :

- The Titanic Survival Prediction project was successfully developed using Python and Machine Learning.
- The Titanic dataset was preprocessed by handling missing values and encoding categorical features.
- A Logistic Regression algorithm was trained to predict whether a passenger would survive based on passenger information.
- The trained model achieved good prediction accuracy and was saved using Joblib for future use.
- A user-friendly frontend was developed using Streamlit, allowing users to enter passenger details and receive instant predictions.
- The application successfully integrates the backend (Google Colab) and frontend (VS Code) into a complete machine learning solution.
- This project demonstrates the complete machine learning lifecycle, including data preprocessing, model training, model evaluation, deployment, and prediction.
- The project enhances understanding of machine learning concepts, data analysis, and web application development using Python.
- The developed system is simple, efficient, and can be further improved by using advanced machine learning algorithms and additional passenger features.
- Overall, the project provides an effective solution for predicting Titanic passenger survival and demonstrates the practical implementation of machine learning in a real-world classification problem.